Remote Device Recorder and Monitoring Lab

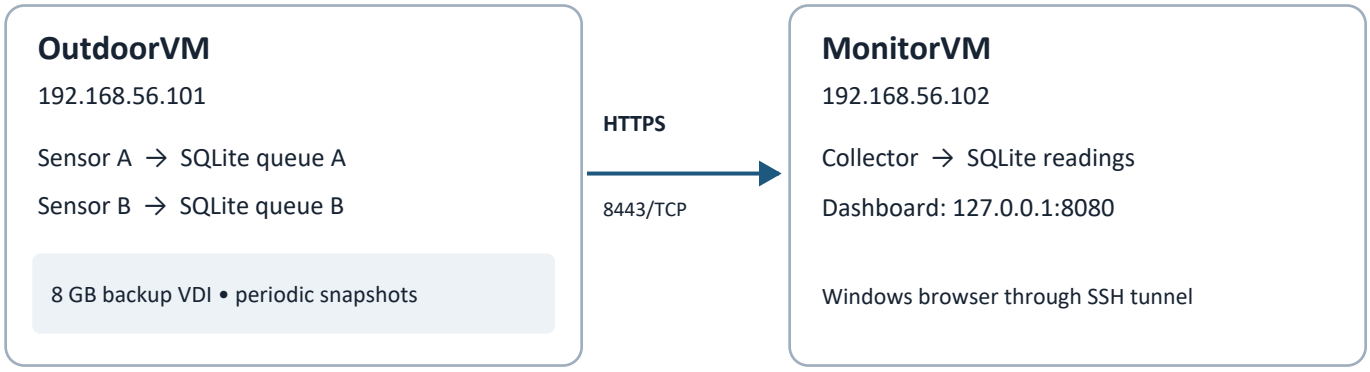
Preserving recorded data when the connection disappears

I deployed two Ubuntu Server virtual machines in VirtualBox to simulate a remote recording station and an office monitoring system. Two sensor processes saved readings locally, queued them during connection outages, and synchronized them after communication returned. A separate virtual disk held scheduled queue backups for a controlled restoration test.

Collector export	Journal reconciliation	Snapshot recovery
879 unique readings	867 of 867 queued IDs found	4 of 4 pending IDs found

Outcome supported by the evidence

The completed lab demonstrates connection-outage buffering, synchronization after connectivity returns, continued reporting from Sensor B while Sensor A is stopped, and restoration of queued readings from a saved backup. Journal and collector reconciliation confirm delivery of all 867 reading IDs recorded as queued in the exported journals.



Host-only network: Windows 192.168.56.1 • NAT provides outbound access

Evidence window: 2 November 2026, 03:17:56–04:45:47 UTC. Screenshot filenames retain the Windows capture date of 1 November. UTC timestamps embedded in the records govern the timeline.

Implementation used supplied Python starter scripts. My project work covered deployment, Linux configuration, execution of failure tests, troubleshooting and evidence collection. This report combines the original screenshots with independent analysis of the uploaded exports.

Build the two recording roles

The outdoor VM hosts the two simulated sensors and their local queues. The monitoring VM receives readings and serves the dashboard. Both guests use NAT plus a host-only adapter; the address plan below reflects the actual lab rather than the example addresses in the original guide.

Component	OutdoorVM	MonitorVM
Role	Sensor simulation and local buffering	Collector and monitoring dashboard
Host-only IPv4	192.168.56.101	192.168.56.102
Memory and CPU	2048 MB / 2 vCPUs	2048 MB / 2 vCPUs
Virtual disks shown	20.62 GB OS + 8.00 GB backup	20.00 GB OS
Guest software	Ubuntu Server; Python; SQLite	Ubuntu Server; Python; SQLite

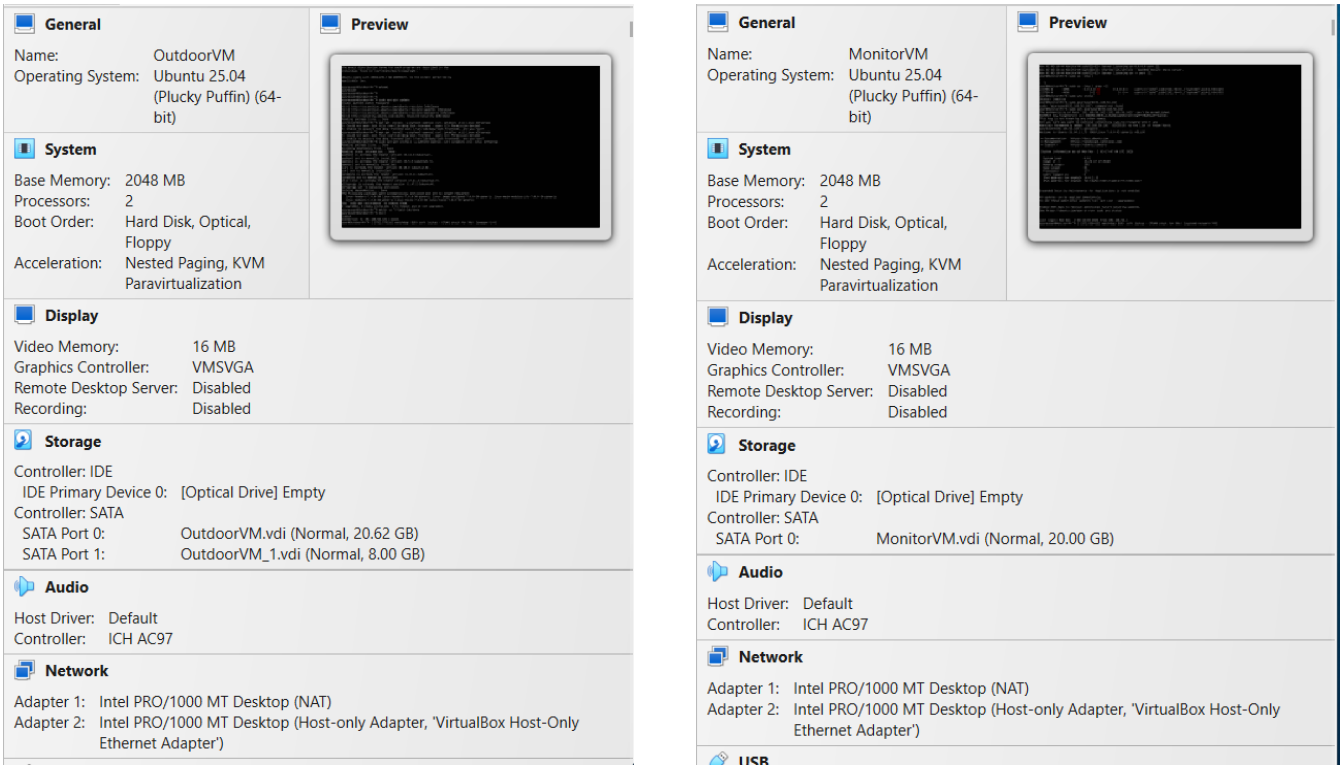


Figure 1. Detail crops from the two VM configurations show distinct roles, CPU/memory settings, storage and NAT/host-only adapters. The unrelated Wazuh VM is outside the report scope. Sources: 204951 and 205029.

Process: create or finish the two guests, enable OpenSSH, attach a new 8 GB VDI only to OutdoorVM, and connect both to the same host-only network. The VirtualBox guest profile displays Ubuntu 25.04; that profile label is not independent verification of the installed OS release.

Separate administration from telemetry

Windows uses 192.168.56.1 on the host-only network to administer the guests over SSH. OutdoorVM sends telemetry to MonitorVM on TCP 8443. The dashboard is reached through an SSH tunnel rather than a publicly exposed web listener.

```
# Windows PowerShell
ssh azureuser@192.168.56.101
ssh azureuser@192.168.56.102

# Each Ubuntu guest
ip -br address
sudo ufw status verbose
```



Figure 2a. OutdoorVM has UFW active with default incoming deny and outgoing allow, and SSH permitted from the Windows host. The uncropped source also records seven offline starter tests completing successfully. Source: 210610.

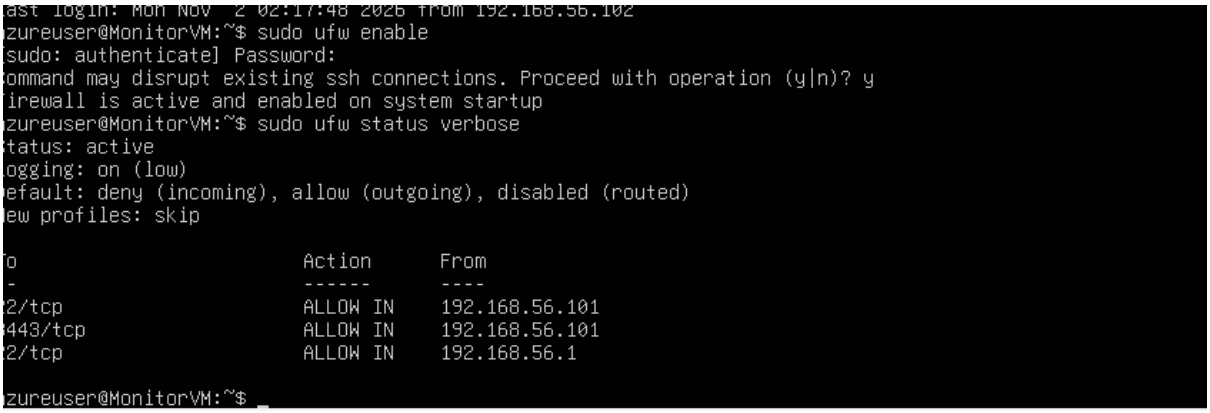


Figure 2b. MonitorVM permits telemetry on 8443 from OutdoorVM and SSH from Windows. An additional SSH allow from OutdoorVM is visible; the configuration therefore allows more than Windows-only administration. Source: 210824.

The screenshots show configured firewall rules, not an exhaustive network penetration test. Actual static-address or DHCP reservation configuration was not supplied; stable address allocation remains a maintenance requirement.

Start collection and establish the baseline

First, prepare Python and the lab directory on each guest and upload the supplied scripts from Windows. Create the monitor certificate with its host-only IP in the Subject Alternative Name, generate a private token, and transfer only the public certificate and shared token to OutdoorVM. Retain the TLS private key on MonitorVM.

```
# Each guest
mkdir -p ~/field-lab/data
cd ~/field-lab
python3 tests.py

# MonitorVM, after certificate and token setup
bash install-services.sh collector
# OutdoorVM: enter 192.168.56.102 at the prompt
bash install-services.sh outdoor
```

Recording and delivery process

Each agent generates a unique reading ID and UTC timestamp, saves the reading to its SQLite outbox, and attempts an authenticated HTTPS delivery. The collector commits accepted IDs before acknowledging them. The agent then removes acknowledged queue entries. Scheduled backups preserve queue state; the collector is the ongoing record after delivery.

```
# Windows: keep this terminal open
ssh -N -L 8080:127.0.0.1:8080 azureuser@192.168.56.102
# Browser: http://localhost:8080
```

Remote station monitoring lab

Two simulated sensors on one outdoor VM. Readings arrive over authenticated HTTPS.

Total stored readings: 196 | Refreshed: 9:41:28 PM

Sensor	Reading freshness	Temperature	Stored	Reported queue*	Latest reading UTC
SENSOR-A	CURRENT	17.98 °C	98	0	2026-11-02T03:41:24.901500+00:00
SENSOR-B	CURRENT	18.05 °C	98	0	2026-11-02T03:41:24.958661+00:00

*Queue size was measured before that reading was queued and sent. It is not a live queue measurement during an outage. Stale means no reading timestamp within 30 seconds; it does not diagnose the cause.

Figure 3. Baseline operation: both sensors CURRENT, 98 readings each and 196 total. Latest recorded timestamps are around 03:41:24 UTC. Source: 214135.

The setup capture records seven successful offline logic tests. The live baseline confirms that both sensor processes deliver readings to the monitoring system.

Keep recoverable queue snapshots

Identify the new empty backup VDI with lsblk and verify its attachment before formatting. The helper creates and mounts an ext4 filesystem at /mnt/field-backup and records a persistent mount. Run an initial backup, then enable the timer. This creates versioned snapshots; it does not mirror every new write continuously.

```
# OutdoorVM: verify the NEW empty disk first
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
# In this lab the verified backup device was /dev/sdb
bash prepare-backup-disk.sh /dev/sdb
python3 backup.py
sudo systemctl enable --now field-backup.timer
findmnt /mnt/field-backup
```

```
azureuser@OutdoorVM:~/field-lab$ df -h ~/field-lab/data /mnt/field-backup
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        21G  3.6G   16G  19% /
/dev/sdb        7.8G  2.1M  7.4G   1% /mnt/field-backup
azureuser@OutdoorVM:~/field-lab$ python3 - <<'PY'
import os
print('Different filesystems:', os.stat('/home/azureuser/field-lab/data').st_dev != os.stat('/mnt/field-backup').st_dev)
PY
Different filesystems: True
azureuser@OutdoorVM:~/field-lab$ python3 backup.py
BACKUP_OK sensor-a-20261102T035017789280Z.sqlite
BACKUP_OK sensor-b-20261102T035017820220Z.sqlite
azureuser@OutdoorVM:~/field-lab$ sudo systemctl enable --now field-backup.timer
Created symlink '/etc/systemd/system/timers.target.wants/field-backup.timer' → '/etc/systemd/system/field-backup.timer'.
azureuser@OutdoorVM:~/field-lab$ systemctl list-timers  field-backup.timer
NEXT                        LEFT LAST                         PASSED UNIT              ACTIVATES
Mon 2026-11-02 03:50:59 UTC   9s Mon 2026-11-02 03:50:29 UTC 20s ago field-backup.timer field-backup.service

1 timers listed.
Pass --all to see loaded but inactive timers, too.
azureuser@OutdoorVM:~/field-lab$ ls -lh /mnt/field-backup/snapshots
total 120K
-rw-r--r-- 1 azureuser azureuser 20K Nov  2 03:50 sensor-a-20261102T035017789280Z.sqlite
-rw------- 1 azureuser azureuser 20K Nov  2 03:50 sensor-a-20261102T035029954932Z.sqlite
-rw------- 1 azureuser azureuser 20K Nov  2 03:51 sensor-a-20261102T035100024063Z.sqlite
-rw-r--r-- 1 azureuser azureuser 20K Nov  2 03:50 sensor-b-20261102T035017820220Z.sqlite
-rw------- 1 azureuser azureuser 20K Nov  2 03:50 sensor-b-20261102T035029981101Z.sqlite
-rw------- 1 azureuser azureuser 20K Nov  2 03:51 sensor-b-20261102T035100057848Z.sqlite
azureuser@OutdoorVM:~/field-lab$ sudo journalctl -u field-backup.service -n 20 --no-pager
Nov 02 03:50:29 OutdoorVM systemd[1]: Starting field-backup.service - Field lab consistent sensor queue snapshots...
Nov 02 03:50:29 OutdoorVM python3[4583]: BACKUP_OK sensor-a-20261102T035029954932Z.sqlite
Nov 02 03:50:29 OutdoorVM python3[4583]: BACKUP_OK sensor-b-20261102T035029981101Z.sqlite
Nov 02 03:50:30 OutdoorVM systemd[1]: field-backup.service: Deactivated successfully.
Nov 02 03:50:30 OutdoorVM systemd[1]: Finished field-backup.service - Field lab consistent sensor queue snapshots.
Nov 02 03:50:59 OutdoorVM systemd[1]: Starting field-backup.service - Field lab consistent sensor queue snapshots...
Nov 02 03:51:00 OutdoorVM python3[4615]: BACKUP_OK sensor-a-20261102T035100024063Z.sqlite
Nov 02 03:51:00 OutdoorVM python3[4615]: BACKUP_OK sensor-b-20261102T035100057848Z.sqlite
```

Figure 4. The primary filesystem is /dev/sda2 and the backup is /dev/sdb. Different filesystems: True, the scheduled timer, snapshot files and BACKUP_OK journal entries confirm the backup workflow. Source: 215358.

The final export reports an approximately 7.8 GiB ext4 backup filesystem with 7.4 GiB available and three snapshot versions per sensor. Backups run approximately every 30 seconds and provide recoverable queue copies; the selected snapshot successfully restored all four pending readings. To further reduce exposure between snapshots, the backup interval can be shortened. A future synchronous-replication extension could acknowledge protection only after each reading is committed to both primary and independent secondary storage.

For stronger fault isolation in a physical deployment, place the secondary copy on a separate physical storage device and verify restoration from that copy. This extends the lab’s working snapshot-and-restore process.

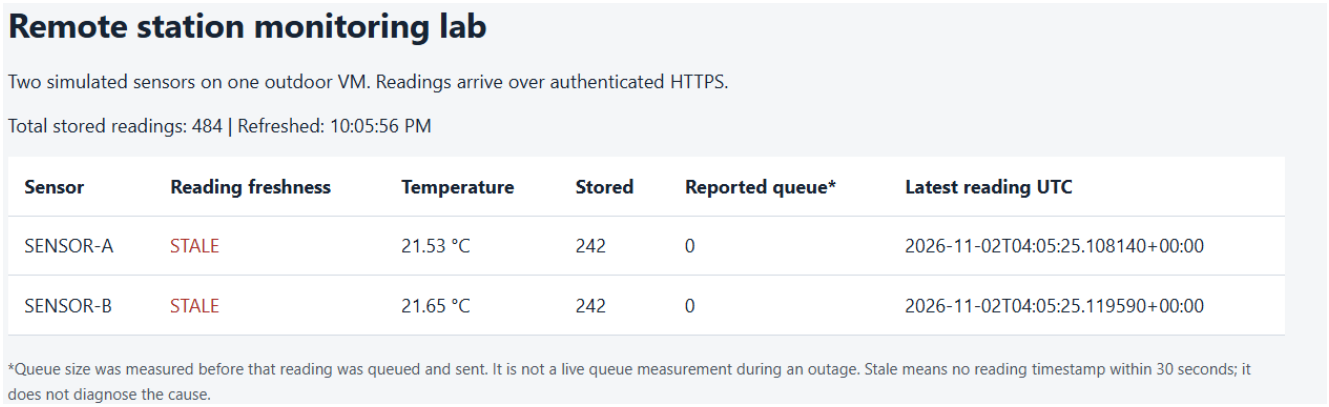
Interrupt delivery while recording continues

Keep both sensors running. In an OutdoorVM terminal, block only telemetry traffic to the monitor. In a second SSH terminal, inspect local queue counters while the dashboard shows stale readings. Let the helper restore the rule, then confirm current readings and drained queues.

```
# OutdoorVM terminal 1 — controlled link test
cd ~/field-lab
bash outage.sh 192.168.56.102 120

# OutdoorVM terminal 2
date -u
python3 agent.py --status --db data/sensor-a.db
python3 agent.py --status --db data/sensor-b.db
```

The 120-second invocation provides a repeatable test window. The exported journals identify the observed delivery failures, queue growth and successful recovery acknowledgement.



Remote station monitoring lab

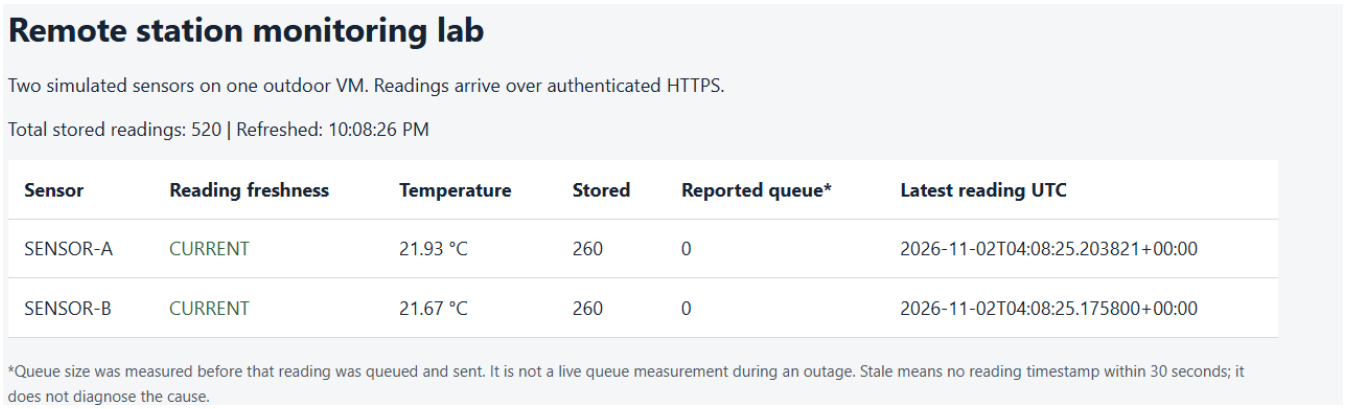
Two simulated sensors on one outdoor VM. Readings arrive over authenticated HTTPS.

Total stored readings: 484 | Refreshed: 10:05:56 PM

Sensor	Reading freshness	Temperature	Stored	Reported queue*	Latest reading UTC
SENSOR-A	STALE	21.53 °C	242	0	2026-11-02T04:05:25.108140+00:00
SENSOR-B	STALE	21.65 °C	242	0	2026-11-02T04:05:25.119590+00:00

*Queue size was measured before that reading was queued and sent. It is not a live queue measurement during an outage. Stale means no reading timestamp within 30 seconds; it does not diagnose the cause.

Figure 5a. During lost delivery both sensors are STALE with 242 stored readings each. Reported queue remains zero because it belongs to the last received sample; it is not a live remote queue measurement. Source: 220604.



Remote station monitoring lab

Two simulated sensors on one outdoor VM. Readings arrive over authenticated HTTPS.

Total stored readings: 520 | Refreshed: 10:08:26 PM

Sensor	Reading freshness	Temperature	Stored	Reported queue*	Latest reading UTC
SENSOR-A	CURRENT	21.93 °C	260	0	2026-11-02T04:08:25.203821+00:00
SENSOR-B	CURRENT	21.67 °C	260	0	2026-11-02T04:08:25.175800+00:00

*Queue size was measured before that reading was queued and sent. It is not a live queue measurement during an outage. Stale means no reading timestamp within 30 seconds; it does not diagnose the cause.

Figure 5b. After recovery both sensors return to CURRENT and the stored counts rise to 260 each. These views support delivery resuming; local queues and journals provide the stronger buffering evidence on the next page. Source: 220835.

Measure the backlog and verify delivery

The local terminal captures pending records rising above zero and later returning to zero. The exported journals add finer detail: both agents reached eight pending records, then each acknowledged a batch of nine at 04:06:55 UTC. The ninth includes a newly sampled reading at recovery.

During interruption

```
azureuser@OutdoorVM:~/field-lab$ python3 agent.py
{
  "generated": 243,
  "acknowledged": 242,
  "rejected": 0,
  "pending": 1
}
azureuser@OutdoorVM:~/field-lab$ python3 agent.py
{
  "generated": 243,
  "acknowledged": 242,
  "rejected": 0,
  "pending": 1
}
```

After backlog drained

```
azureuser@OutdoorVM:~/field-lab$ python3 agent.py
{
  "generated": 258,
  "acknowledged": 258,
  "rejected": 0,
  "pending": 0
}
azureuser@OutdoorVM:~/field-lab$ python3 agent.py
{
  "generated": 259,
  "acknowledged": 259,
  "rejected": 0,
  "pending": 0
}
```

Figure 6. Two detail crops from the same terminal capture show each sensor with pending 1 during interruption and pending 0 afterward. Rejected remains 0. Commands extend beyond the crop; all cited counter values remain visible. Source: 221020.

Measured sequence: first logged failure at 04:05:38; pending reaches 8 at 04:06:48; acknowledged=9 and pending=0 at 04:06:55. The 77 seconds from the first failure to success includes detection and retry timing, rather than directly measuring the link outage.

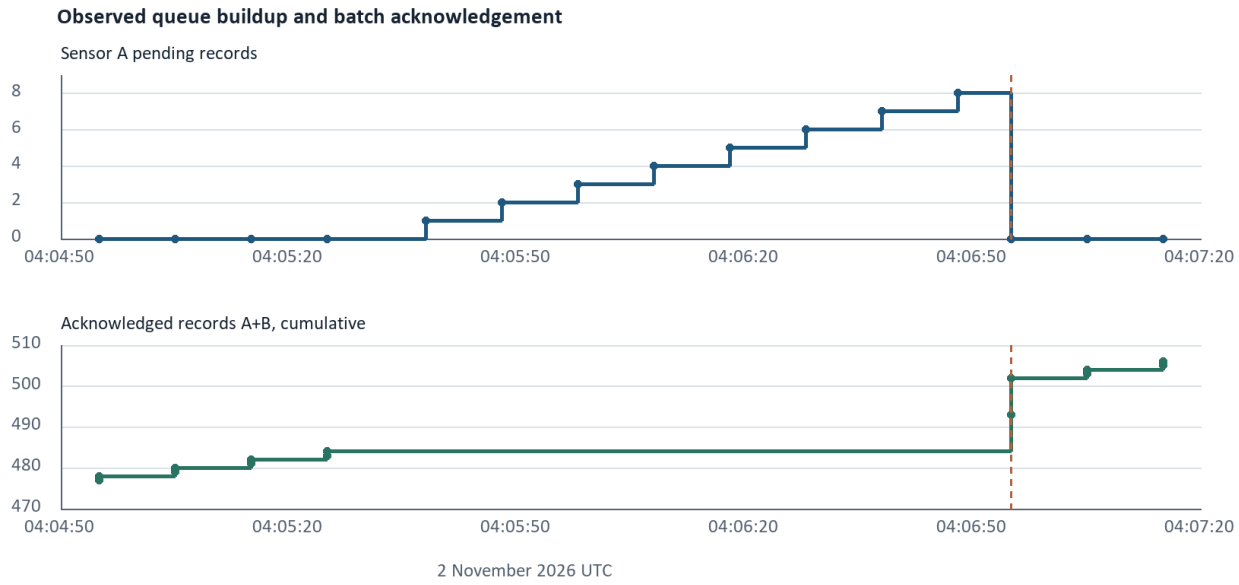


Figure 7. Derived from journal timestamps and counters. The upper trace shows Sensor A queue depth; Sensor B follows the same values in this interval. The lower trace sums acknowledgement counts from both agents and shows the recovery batch. Dashed line: 04:06:55 UTC.

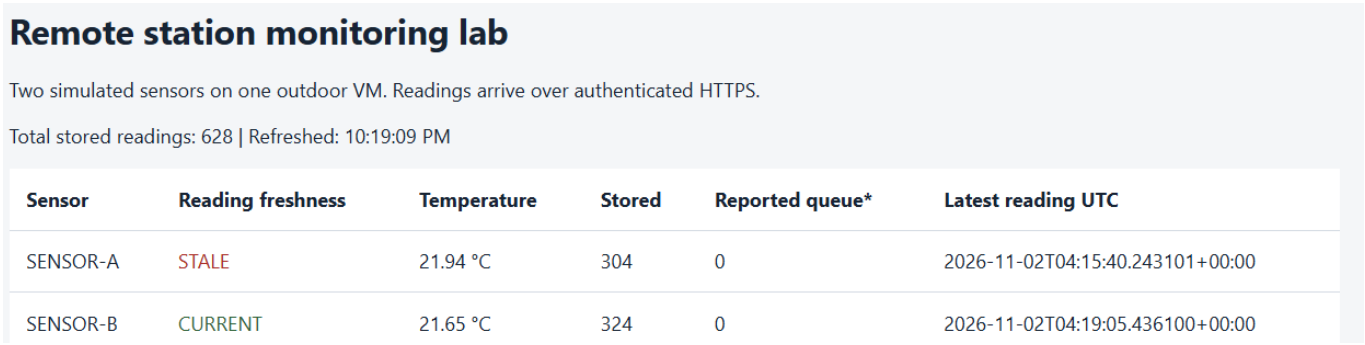
At the outdoor export, generated and acknowledged counts are equal for both sensors and pending/rejected are zero. Independent ID reconciliation found every one of the 867 unique IDs logged as queued in the supplied journals in the later collector CSV.

Keep one sensor reporting when another stops

Stop only Sensor A and keep Sensor B running. After the freshness threshold passes, compare their dashboard states, then restart A. The result demonstrates process-level continuity: Sensor B continues reporting while Sensor A is interrupted, and Sensor A resumes reporting after it starts again.

```
# OutdoorVM
sudo systemctl stop field-sensor-a
# Wait at least 35 seconds; inspect the dashboard
sudo systemctl start field-sensor-a

# Separate process-crash test
sudo systemctl kill --kill-whom=main --signal=SIGKILL \
  field-sensor-a.service
sudo journalctl -u field-sensor-a -n 30 --no-pager
```



Remote station monitoring lab

Two simulated sensors on one outdoor VM. Readings arrive over authenticated HTTPS.

Total stored readings: 628 | Refreshed: 10:19:09 PM

Sensor	Reading freshness	Temperature	Stored	Reported queue*	Latest reading UTC
SENSOR-A	STALE	21.94 °C	304	0	2026-11-02T04:15:40.243101+00:00
SENSOR-B	CURRENT	21.65 °C	324	0	2026-11-02T04:19:05.436100+00:00

Figure 8. Sensor A is STALE at 304 stored readings while B remains CURRENT at 324. Their last-recorded timestamps differ, showing that B continued reporting during A's interruption. Source: 221917.

What the journal confirms

UTC event	Observed result
04:13:06 → 04:13:12	Sensor A killed with signal 9, then service started again: 6 seconds.
04:14:24 → 04:14:29	A second signal-9 termination is followed by restart: 5 seconds.
04:15:44 → 04:19:37	Sensor A intentionally stopped and subsequently started. Sensor B has no matching stop during this interval.

An intentional systemctl stop does not trigger the same automatic restart path as an unexpected process failure. The logs support both distinctions. A later status export shows NRestarts=0 for the current service invocation; that final value does not erase the historical restart events in the journal.

Restore a saved queue and check its IDs

During a separate connectivity interruption, pause both sensors and the backup timer so the recovery point stays stable. Take a consistent snapshot, preserve the original queue file rather than deleting it, and restore Sensor A to the original database path. Restart collection and restore connectivity.

```
# OutdoorVM, while telemetry is blocked
sudo systemctl stop field-sensor-a field-sensor-b
sudo systemctl stop field-backup.timer field-backup.service
python3 backup.py
# Select the latest A snapshot and preserve it as:
# recovery-evidence.sqlite
# Keep the original primary database before restoring
mv -n data/sensor-a.db data/sensor-a.before-restore.db
python3 recover.py --snapshot recovery-evidence.sqlite \
  --output data/sensor-a.db
sudo systemctl start field-sensor-a field-sensor-b
sudo systemctl start field-backup.timer
```

Procedure excerpt: the full guide supplies snapshot selection and file-preservation checks. The restore tool preserves existing destination files by requiring a new output path.

Inspect the uploaded recovery point

Snapshot property	Observed value
Integrity check	ok
Counters	generated 377; acknowledged 373; rejected 0
Pending records	4 Sensor A readings, recorded 04:33:22–04:33:52 UTC
CSV ID reconciliation	All 4 snapshot pending IDs found; missing 0

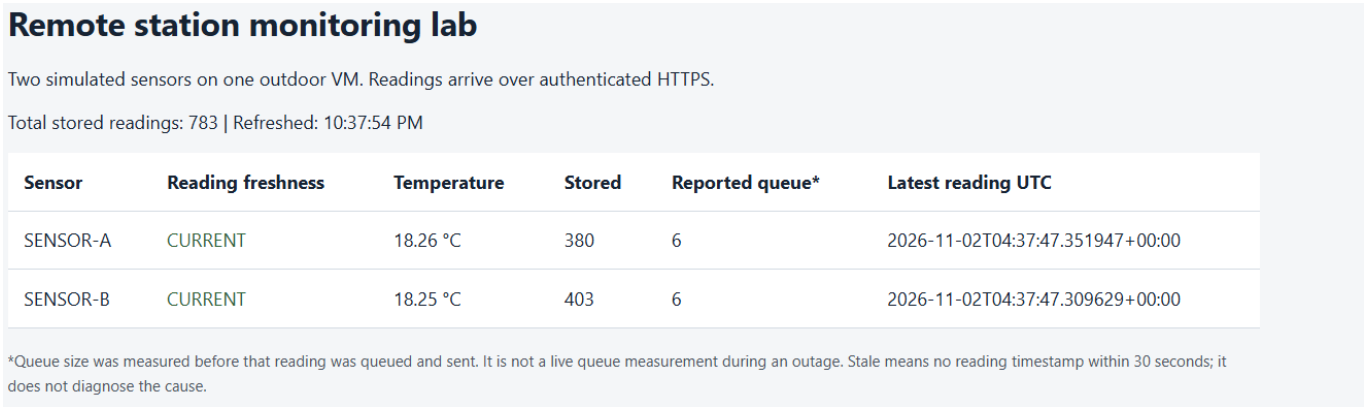
```
# MonitorVM, after transferring the selected snapshot
python3 verify_recovery.py --snapshot recovery-evidence.sqlite
```

```
azureuser@MonitorVM:~/field-lab$ python3 verify_recovery.py --snapshot recovery-evidence.sqlite
Snapshot pending IDs: 4; found: 4; missing: 0
azureuser@MonitorVM:~/field-lab$
```

Figure 9. The live verification reports Snapshot pending IDs: 4; found: 4; missing: 0. Independent read-only analysis of the uploaded SQLite snapshot and CSV reproduces that result. Source: 224250.

Observe synchronization settling back to normal

The recovery screenshots capture two different stages and are both useful. The first records the historical backlog reported by a newly received sample. The next shows a subsequent reading with no queue reported. Neither image alone is an instantaneous view of remote disk state.



Remote station monitoring lab

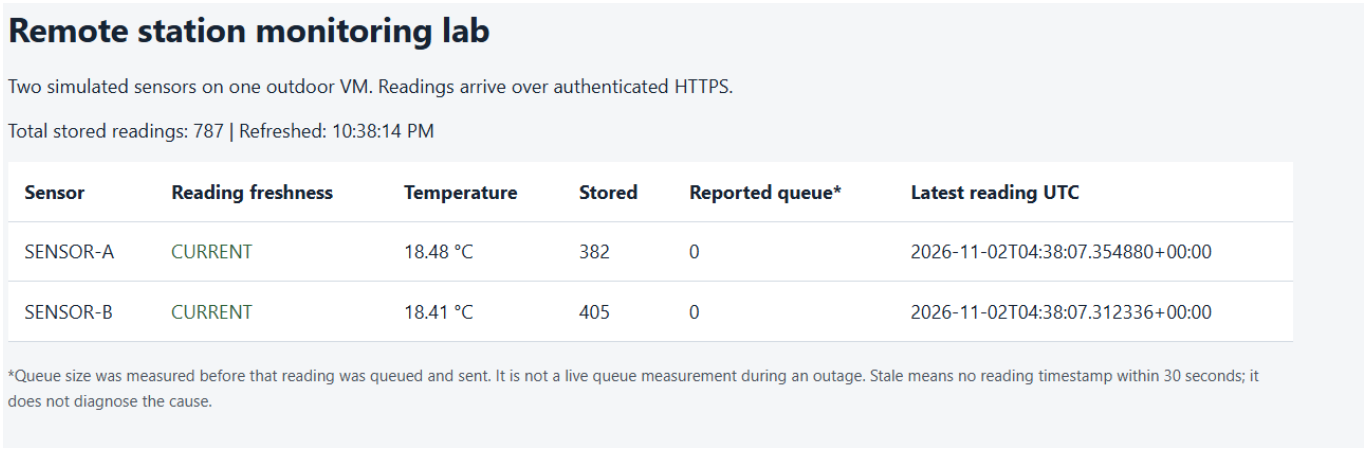
Two simulated sensors on one outdoor VM. Readings arrive over authenticated HTTPS.

Total stored readings: 783 | Refreshed: 10:37:54 PM

Sensor	Reading freshness	Temperature	Stored	Reported queue*	Latest reading UTC
SENSOR-A	CURRENT	18.26 °C	380	6	2026-11-02T04:37:47.351947+00:00
SENSOR-B	CURRENT	18.25 °C	403	6	2026-11-02T04:37:47.309629+00:00

*Queue size was measured before that reading was queued and sent. It is not a live queue measurement during an outage. Stale means no reading timestamp within 30 seconds; it does not diagnose the cause.

Figure 10a. After communication returns, both sensors are CURRENT and each latest sample reports a prior queue of 6. The collector has 783 total readings. Source: 223804.



Remote station monitoring lab

Two simulated sensors on one outdoor VM. Readings arrive over authenticated HTTPS.

Total stored readings: 787 | Refreshed: 10:38:14 PM

Sensor	Reading freshness	Temperature	Stored	Reported queue*	Latest reading UTC
SENSOR-A	CURRENT	18.48 °C	382	0	2026-11-02T04:38:07.354880+00:00
SENSOR-B	CURRENT	18.41 °C	405	0	2026-11-02T04:38:07.312336+00:00

*Queue size was measured before that reading was queued and sent. It is not a live queue measurement during an outage. Stale means no reading timestamp within 30 seconds; it does not diagnose the cause.

Figure 10b. Twenty seconds later in the displayed refresh times, subsequent readings report queue 0 and the total has increased to 787. Source: 223818.

The journals place both sensor stops at 04:33:57 UTC and both starts at 04:37:27. At 04:37:47 each acknowledges seven records and has pending=0. This is consistent with delivery of buffered records after the controlled restoration sequence.

The selected snapshot contains four recoverable pending readings. Pausing the processes establishes a stable recovery point, and restarting them resumes sampling and synchronization. Independent matching confirms that all four snapshot IDs reached the collector.

Reconcile the different observation times

The outdoor and monitoring exports were taken at different times while collection was still running. Compare each observation at its own timestamp; their totals are not contradictory.

Evidence snapshot	Sensor A	Sensor B	Total
Outdoor counters at 04:44:57 UTC	422 generated / acknowledged	445 generated / acknowledged	867
Outdoor pending and rejected	0 / 0	0 / 0	0 pending
Monitor CSV through 04:45:47 UTC	428 stored	451 stored	879
Additional later CSV records	6	6	12
Queued journal IDs found in CSV	422 of 422	445 of 445	867 of 867

Independent checks performed for this report

The CSV contains 879 rows and 879 unique reading IDs. Every queued=True ID extracted from the provided sensor journals is present in that later CSV. The uploaded recovery snapshot passes SQLite integrity_check, and all four pending IDs match CSV records. The final dashboard JSON agrees with the CSV totals: 428 for A and 451 for B.

Source sets: outdoor-20261102T044457260600Z and monitor-20261102T0445480425 27Z. Exported counters precede the end of the collector CSV by approximately one minute of sampling.

Successful validation results

Test or property	Assessment	Evidence
Baseline sensor delivery	Demonstrated	Both sensors CURRENT; baseline screenshot and collector records.
Outage buffering and recovery	Demonstrated	Failed deliveries, queues up to 8 each, then 9-record acknowledgements.
Single sensor interruption	Demonstrated	A STALE while B CURRENT; matching service events.
Automatic process restart	Demonstrated	Two signal-9 terminations followed by starts after 6 and 5 seconds.
Backup creation and restore	Demonstrated	Separate filesystem, snapshots, 4 of 4 pending IDs recovered.
VM reboot recovery	Successful	Successful reboot recovery confirmed by the project author.

Reproduce the workflow with the actual addresses

These commands summarize the process using the lab's observed address plan. Use the reproduction reference alongside the setup guide, verify new disks before formatting, and preserve prior recovery files.

1 Prepare and transfer

```
# Each guest, once OpenSSH and Python are installed
mkdir -p ~/field-lab/data

# Windows PowerShell in the script folder
$OutdoorIP = "192.168.56.101"
$MonitorIP = "192.168.56.102"
scp .\*.py .\*.sh "azureuser@${OutdoorIP}:/home/azureuser/field-lab/"
scp .\*.py .\*.sh "azureuser@${MonitorIP}:/home/azureuser/field-lab/"
```

2 Configure and validate

```
# MonitorVM: generate certificate with IP SAN
MONITOR_PRIVATE=192.168.56.102
openssl req -x509 -newkey rsa:2048 -sha256 -days 30 -nodes \
  -keyout collector.key -out collector.crt \
  -subj /CN=field-lab-monitor \
  -addext "subjectAltName=IP:${MONITOR_PRIVATE}"
chmod 600 collector.key
openssl rand -hex 32 > token.txt
chmod 600 token.txt
bash install-services.sh collector
```

Transfer collector.crt and token.txt to OutdoorVM through the Windows host; keep collector.key on MonitorVM. Remove the temporary Windows token copy after successful transfer. On OutdoorVM run install-services.sh outdoor and enter 192.168.56.102. Verify service status and the dashboard before injecting failures.

3 Capture and export before shutting down

```
# OutdoorVM
sudo -v
python3 export_evidence.py outdoor
# MonitorVM
sudo -v
python3 export_evidence.py monitor
# Download and open BOTH archives locally. Then on each VM:
sudo poweroff
```

Keep the evidence traceable

Source screenshot suffix	Use in this report
204951 / 205029	VM hardware, storage and network detail crops; Figure 1.
210610 / 210824	Offline tests and active Linux firewall rules; Figures 2a–2b.
214135	Baseline dashboard; Figure 3.
215358	Mounted backup disk, timer and snapshots; Figure 4.
220604 / 220835	Stale and recovered delivery views; Figures 5a–5b.
221020	Local queue counters; Figure 6.
221917	Sensor A stale while B remains current; Figure 8.
224250	Recovery verification terminal; Figure 9.
223804 / 223818	Post-restoration queue-report sequence; Figures 10a–10b.

All screenshot filenames use the prefix Screenshot 2026-11-01 and suffix .png. The six-digit entries above identify the original files exactly within that set. Figure 7 is newly plotted from the supplied journals, not a screenshot.

Data and journal sources

Outdoor export: queue-counters.json; field-sensor-a-journal.txt; field-sensor-b-journal.txt; backup-disk.txt; field-backup.service-journal.txt; field-backup.timer-journal.txt; service status files.
Monitor export: readings.csv; dashboard-status.json; field-collector-journal.txt; field-dashboard-journal.txt; service status files.
Recovery evidence: recovery-evidence.sqlite, examined read-only for integrity, counters and pending IDs.

Project statement

Built and tested a two-VM Ubuntu monitoring lab in VirtualBox using supplied Python starter scripts, with dual simulated sensors, persistent local queues and scheduled backups on a separate virtual disk. Verified connection-outage recovery, sensor-process continuity and restoration of four queued readings from a selected backup.